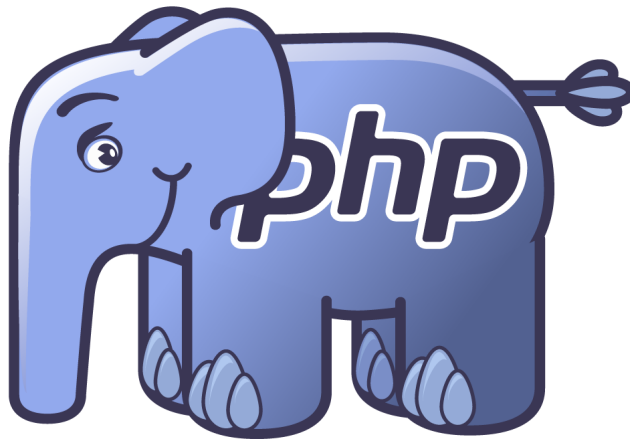


Introduction au PHP



NSI Première

Académie de Versailles

M. Boehm, C. Poulmaire, P. Remy

Sommaire

1	Quelques règles générales sur le langage PHP	2
2	Instructions, variables, commentaires et sorties	2
3	Opérations et fonctions mathématiques	3
3.1	Les opérations usuelles	3
3.2	Fonctions mathématiques	3
4	Structures de données	3
4.1	Les types	3
4.2	Les structures booléennes	4
4.3	Les chaînes de caractères	4
4.4	Les tableaux	5
4.5	Les fichiers textes	6
5	Structures de contrôles	6
5.1	Les tests	6
5.1.1	Les opérateurs de comparaison	6
5.1.2	Les opérateurs logiques	6
5.2	Les structures d’embranchements	7
5.2.1	La structure <code>si ... alors</code>	7
5.2.2	La structure <code>si ... alors ... sinon</code>	7
5.2.3	Les structures d’embranchements multiples	8
5.3	La structure de boucle <code>pour</code>	9
5.4	La structure de boucle <code>tant que</code>	9
6	Les fonctions	10
7	Liaisons entre les fichiers PHP	10
8	Dates et heures	11
9	Gestion des formulaires	12

Le principe général du langage PHP (= PHP :Hypertext Preprocessor; acronyme récursif) est de créer des sites web dynamiques, c'est-à-dire des sites web qui puissent afficher du contenu changeant sans l'intervention d'un développeur pour en modifier le code. C'est un langage de programmation libre créé en 1994 par Rasmus Lerdorf de la famille des langage impératif orienté objet, mais qui peut aussi fonctionner comme n'importe quel langage interprété de façon locale.

1 Quelques règles générales sur le langage PHP

- ★ Pour faire fonctionner un site dynamique écrit à l'aide de PHP, l'ordinateur a besoin d'un serveur web (distant ou interne).
- ★ Un code source PHP peut être inséré n'importe où dans un code HTML.
- ★ Un code PHP soit être inséré entre la balise ouvrante `<?php` et la balise fermante `?>`.
- ★ Un fichier contenant un code PHP doit obligatoirement être sauvegardé avec l'extension `.php`.

Le langage PHP est avant tout un langage de programmation (comme Python, C, etc...). On va donc manipuler des variables, des structures d'embranchements, de boucles, etc...

2 Instructions, variables, commentaires et sorties

- ★ **Une instruction** se termine toujours par un point-virgule ;
- ★ **Le nom d'une variable** commence toujours par le symbole `$`
- ★ **Affecter une valeur à une variable `x`** : `$x= ...`
- ★ **Cloner un objet** : `clone $objet`
- ★ **Afficher à l'écran du texte** : `echo "texte";` ou `echo 'texte';`
On retrouve ici les guillemets doubles `"..."` et guillemets simples `'...'` permettant de définir une chaîne de caractères.
 - Si une variable `$x` est écrite entre des guillemets doubles, elle est remplacée par sa valeur (sauf si elle correspond à un tableau ou à une fonction). Ceci permet en particulier d'afficher son contenu. En revanche, si elle est écrite entre des guillemets simples, elle n'est pas évaluée.
 - Le texte étant bien sûr affiché dans une page HTML, celui-ci est écrit avec la syntaxe HTML. On peut donc utiliser en particulier toutes les balises HTML classiques.
 - Pour afficher des guillemets, on utilise comme en JavaScript le caractère d'échappement `\`, ce qui donne `"`.
 - Tant que l'on n'a pas refermé le guillemet `"`, on peut passer à la ligne afin d'écrire le texte sur plusieurs lignes.
- ★ **Les commentaires**, qui ne sont pas pris en compte dans l'exécution du script, doivent toujours être précédés du symbole `//` (lorsqu'ils sont situés sur une seule ligne). Pour afficher plusieurs lignes successives de commentaires, on les encadre par les commandes `/*` et `*/` comme ceci : `/* on tape les commentaires */`.

Exemple 2.1 Le code PHP suivant permet d'affecter le nom Pierre à la variable `$nom` et la valeur 17 à la variable `$x`, puis de les afficher, le nom étant entre guillemets :

```
<?php // on ouvre la balise php
    $nom="Pierre"; // on affecte Pierre
    $x=17; // on affecte la valeur 17
    echo "<h1>Bonjour !!!</h1>";
    echo "<p>Je m'appelle \"\$nom\" et
        j'ai $x ans.</p>";
?> // on ferme la balise php
```

Même s'il y a deux instructions `echo`, il n'y a pas de saut de ligne à l'affichage. En effet, le saut de ligne ne peut être effectif qu'avec l'utilisation de la balise `<br/>` du langage HTML.

3 Opérations et fonctions mathématiques

3.1 Les opérations usuelles

★ Opérations algébriques.

addition	soustraction	multiplication	division	puissance $n^{\text{ème}}$ de x
+	−	*	/	$x ** n$

Remarque 3.1 Attention, les opérations doivent toujours être écrites. Par exemple, on écrit $2 * x$ et non pas $2x$.

★ Opérations avec les nombres entiers. Soient a et b deux nombres entiers.

quotient de a par b	reste de a par b
<code>intdiv(a, b)</code>	$a \% b$

Exemple 3.2 L'instruction `intdiv(5, 2)` renvoie 2 et l'instruction `5%2` renvoie 1.

★ Valeur absolue et arrondis.

valeur absolue de x	arrondi de x avec n décimales
<code>abs(x)</code>	<code>round(x, n)</code>

3.2 Fonctions mathématiques

Un certain nombre de fonctions mathématiques usuelles sont implémentées en PHP. Pour les utiliser, il est nécessaire d'importer la bibliothèque `math` :

★ Fonctions arithmétiques.

- `floor( $x$ )` : renvoie la partie entière (inférieure) de x .
- `ceil( $x$ )` : renvoie la partie entière supérieure de x .

★ Exponentielle et logarithmes.

- `exp( $x$ )` : renvoie l'exponentielle de x .
- `log( $x$ )` : renvoie le logarithme népérien de x .
- `sqrt( $x$ )` : renvoie la racine carrée de x .
- `pow( $x$ ,  $y$ )` : renvoie la puissance de x par y .

★ Valeurs aléatoires.

- `rand( $a$ ,  $b$ )` : renvoie un nombre réel aléatoire dans l'intervalle $[a; b]$.

4 Structures de données

4.1 Les types

★ `gettype( $x$ )` : renvoie le type de la variable x . On a les correspondances suivantes :

entier	flottant	chaîne de caractères	tableau	booléen
<code>integer</code>	<code>double</code>	<code>string</code>	<code>array</code>	<code>boolean</code>

Lorsque la fonction `gettype()` ne reconnaît pas le type de la variable, elle renvoie la chaîne `unknown type`. D'autre part, il existe un grand nombre de fonctions qui permettent de vérifier si une variable est d'un type donné ou non :

- `is_int( $x$ )` : renvoie `TRUE` si la variable x est un entier et `FALSE` sinon,
 - `is_float( $x$ )` : renvoie `TRUE` si la variable x est un flottant et `FALSE` sinon,
 - `is_numeric( $x$ )` : renvoie `TRUE` si la variable x est une chaîne numérique et `FALSE` sinon,
 - `is_string( $x$ )` : renvoie `TRUE` si la variable x est une chaîne de caractères et `FALSE` sinon,
 - `is_array( $x$ )` : renvoie `TRUE` si la variable x est un tableau et `FALSE` sinon,
 - `is_bool( $x$ )` : renvoie `TRUE` si la variable x est un booléen et `FALSE` sinon.
- ★ `settype( $x$ ,  $type$ )` : permet de transtyper, si cela est possible, la variable x en le type $type$ qui est donné par l'une des chaînes de caractères du tableau précédent.
- ★ Pour transtyper les variables, on peut aussi indiquer avec la variable le type désiré entre parenthèses. Par exemple, si $x=5.2$, alors `$y=(integer) x$` est un entier valant 5.
- ★ `var_dump( $x$ )` : affiche les informations structurées de la variable x .

4.2 Les structures booléennes

Les données de type `boolean` sont des données qui ne peuvent prendre que deux valeurs : `True` ou `False`.

- ★ Ce type de données apparaît lorsque l'on teste si une condition est réalisée. Par exemple, l'instruction `2 < 8` renvoie la valeur `True`, alors que l'instruction `5 < 1` renvoie la valeur `False`.
- ★ Il peut être également intéressant d'affecter directement des données booléennes à des variables, notamment pour définir des instructions bloquantes à l'aide d'une boucle `while`.

4.3 Les chaînes de caractères

Pour un fichier purement PHP, il est important de déclarer le type d'encodage utilisé avec l'instruction

```
header('content-type:text/html;charset=nom_encodage')
```

Pour nous, `nom_encodage` sera `utf-8` ou `ansi`. N'oubliez pas aussi de vérifier l'encodage du fichier.

- ★ Une chaîne de caractères est définie entre des guillemets doubles `" . . . "` ou des guillemets simples `' . . . '`.
- ★ Si une variable `$x` est écrite entre des guillemets doubles, elle est remplacée par sa valeur (sauf si elle correspond à un tableau ou à une fonction). En revanche, si elle est écrite entre des guillemets simples, elle n'est pas évaluée.
- ★ Pour concaténer deux chaînes de caractères, on utilise l'opérateur `.`

Exemple 4.1 L'instruction

```
"Bonjour"." à vous"
```

renvoie la chaîne de caractères `"Bonjour à vous"`.

- ★ Quelques fonctions utiles sur les chaînes de caractères :
 - `strlen($chn)` : renvoie le nombre de caractères de la chaîne `$chn`.
 - `$chn[i]` : renvoie le caractère de la chaîne `$chn` situé au rang `i+1` (le premier caractère est donc indexé par 0).
 - `strtoupper($chn)` : renvoie une copie en majuscule de la chaîne `$chn`, cette dernière n'étant pas modifiée.
 - `strtolower($chn)` : renvoie une copie en minuscule de la chaîne `$chn`, cette dernière n'étant pas modifiée.
 - `strpos($chn, $car)` : renvoie la position de la première occurrence de la sous-chaîne `$car` dans la chaîne `$chn` si elle existe et `FALSE` sinon.
 - `substr($chn, $deb, $nbr)` : renvoie la sous-chaîne commençant à la position `$deb` dans la chaîne `$chn` et contenant `$nbr` caractères.
 - `str_replace($old, $new, $chn)` : renvoie la chaîne de caractères dans laquelle toutes les occurrences de la chaîne `$chn` définies par `$old` ont été remplacées par la chaîne `$new`; la chaîne initiale n'est pas modifiée.
 - `trim($chn)` : renvoie une copie de la chaîne de caractères `$chn` après avoir supprimé le caractères invisibles au début et à la fin (espace, tabulation, passage à la ligne, retour chariot, tabulation verticale, caractère NUL). On peut rajouter le paramètre optionnel `character_mask` pour spécifier à l'aide d'une chaîne de caractères les caractères à supprimer; on peut utiliser `.` pour en indiquer plusieurs.

Exemple 4.2 On considère la chaîne de caractères `$chn = 'Bonjour'`. Alors :

- `strlen($chn)` renvoie la valeur 6;
- `$chn[1]` renvoie la chaîne de caractères `'o'`;
- `strtolower($chn)` renvoie la chaîne de caractères `'bonjour'`;
- `str_replace('jour', 'bon', $chn)` renvoie la chaîne de caractères `'Bonbon'`.

Remarque 4.3 Dans un encodage `UTF-8`, les caractères accentués comptent pour 2 caractères puisqu'ils sont encodés sur 2 octets. En revanche, dans un encodage `ANSI`, ils ne comptent que pour 1 caractère car ils sont encodés sur 1 octet.

Remarque 4.4 Une chaîne de caractères peut être parcourue caractère par caractère à l'aide d'une boucle `for` sur les index.

La fonction `explode($sep, $chn)` renvoie un tableau indexé numériquement contenant les éléments de la chaîne `$chn` définis à l'aide du séparateur `sep` (c'est l'analogue de `split` en Python et JavaScript).

4.4 Les tableaux

Les tableaux PHP sont des données structurées de type `array`. Il en existe de deux sortes : le tableau vide (qui sert en général à initialiser une variable) et les tableaux associatifs (l'équivalent des dictionnaires en Python). Parmi les tableaux associatifs, il existe le cas particulier des tableaux indexés numériquement (l'équivalent des listes en Python).

- ★ `$x = array()` : affecte à la variable `$x` un tableau vide.
- ★ `$x = array(cle1=>val1, cle2=>val2, ..., clen=>valn)` : affecte à la variable `$x` un tableau associatif de paire clé; valeur. Les clés sont entières ou des chaînes de caractères et les valeurs peuvent être de n'importe quel type. Pour un tel tableau :
 - `count($x)` ou `sizeof($x)` : renvoie le nombre d'éléments du tableau `$x`.
 - `$x[cle]` : renvoie la valeur du tableau `$x` correspondant à la clé `cle` si elle est présente dans le tableau.
 - `$x[cle] = val` : ajoute au tableau `$x` la clé `cle` associée à la valeur `val` si la clé n'existe pas; modifie la valeur existante par `val` sinon.
 - `$x[] = val` : ajoute au tableau `$x` la valeur `val` associée à la clé « valeur numérique maximale présente + 1 » s'il existe des clés numériques; 0 sinon.
 - `unset($x[cle])` : efface l'élément du tableau `$x` de clé `cle`.
 - `unset($x)` : efface le tableau `$x`.
 - `array_key_exists(cle, $x)` : renvoie `TRUE` si la clé `cle` est présente dans le tableau `$x` et `FALSE` sinon.
 - `array_keys($x)` : renvoie un tableau contenant toutes les clés du tableau `$x`; ce tableau est indexé par les valeurs numériques 0, 1, etc...

Exemple 4.5 On considère le tableau `$T = array('orange'=>40, 'poire'=>10)`. Alors :

- `count($T)` renvoie la valeur 2;
 - `$T['orange']` renvoie la valeur 40;
 - `$T['orange']=50` modifie le tableau en `array('orange'=>50, 'poire'=>10)`;
 - `$T['pomme']=5` modifie le tableau en `array('orange'=>50, 'poire'=>10, 'pomme'=>5)`;
 - `array_key_exists('pomme', $T)` renvoie `TRUE`;
 - `unset($T['pomme'])` modifie le tableau en `array('orange'=>50, 'poire'=>10)`;
 - `array_key_exists('pomme', $T)` renvoie `FALSE`.
-
- ★ `$x = array(val1, val2, ..., valn)` : affecte à la variable `$x` un tableau indexé numériquement de valeurs `val1, val2, ..., valn`. Les valeurs peuvent être de n'importe quel type. Ce tableau est en fait un tableau associatif dont les clés sont les index 0, 1, ... Pour un tel tableau :
 - Toutes les instructions précédentes sont valables. En particulier :
 - `$x[] = val` : ajoute à la fin du tableau `$x` la valeur `val`.
 - `$x[i]` : renvoie l'élément du tableau `$x` situé à l'index `i`.
 - `$x[i] = val` : remplace la valeur d'index `i` du tableau `$x` par la valeur `val`.
 - `unset($x[i])` : efface l'élément du tableau situé à l'index `i`, mais sans ré-indexation. Pour ce faire, il faut utiliser l'instruction `$x = array_values($x)`.
 - Le tableau `$x` peut être parcouru élément par élément à l'aide d'une boucle `for` sur les index.

Exemple 4.6 On considère le tableau `$T = array('orange', 'poire')`. Alors :

- `count($T)` renvoie la valeur 2;
- `$T[0]` renvoie la valeur 'orange' ;
- `$T[0]=50` modifie le tableau en `array(50, 'poire')` ;
- `$T[]='pomme'` modifie le tableau en `array(50, 'poire', 'pomme')`.

4.5 Les fichiers textes

★ Ouverture et fermeture d'un fichier texte.

- `file_exists(nom_fichier)` : renvoie `TRUE` si le fichier donné par la chaîne de caractères `nom_fichier` existe et `FALSE` sinon.
- `$F=fopen(nom_fichier,mode)` : ouvre le fichier donné par la chaîne de caractères `nom_fichier` avec le mode `mode` et l'affecte à la variable `$F`. On a les modes classiques suivants :
 - `r` : ouverture en lecture seule ; si le fichier n'existe pas, il y a une erreur.
 - `a` : ouverture en ajout seul ; si le fichier n'existe pas, il est créé.
 - `w` : ouverture en écriture seule ; si le fichier n'existe pas, il est créé et si le fichier existe, ses données sont supprimées.
 Il existe d'autres types de mode d'ouverture (`r+`, `a+`, `w+`, etc...).
- `fclose($F)` : ferme le fichier contenu dans la variable `$F`.

★ Lecture d'un fichier texte.

- `is_readable(nom_fichier)` : renvoie `TRUE` si le fichier donné par la chaîne de caractères `nom_fichier` existe et est accessible en lecture ; et `FALSE` sinon.
- `fgetc($F)` : permet de lire le fichier contenu dans la variable `$F` caractère par caractère.
- `fgets($F)` : permet de lire le fichier contenu dans la variable `$F` ligne par ligne.
- `fread($F, filesize(nom_fichier))` : renvoie l'intégralité du fichier donné par la chaîne de caractères `nom_fichier` et contenu dans la variable `$F`.

★ Ecriture dans un fichier texte.

- `is_writable(nom_fichier)` : renvoie `TRUE` si le fichier donné par la chaîne de caractères `nom_fichier` existe et est accessible en écriture ; et `FALSE` sinon.
- `fwrite($F, texte)` : permet d'écrire la chaîne de caractères `texte` dans le fichier contenu dans la variable `$F`.
- `fputs($F, texte)` : alias de `fwrite()`.

5 Structures de contrôles

5.1 Les tests

5.1.1 Les opérateurs de comparaison

égal	différent	inférieur	supérieur	inférieur strict	supérieur strict
<code>==</code>	<code>!=</code>	<code><=</code>	<code>>=</code>	<code><</code>	<code>></code>

Exemple 5.1 Les instructions

```
3 == 5; "ab" != "cd"; 5 <= 8; 4 >= -3; 3 < 1;
```

renvoient respectivement `False`, `True`, `True`, `True` et `False`.

5.1.2 Les opérateurs logiques

ou	ou exclusif	et	non
<code>OR</code>	<code>XOR</code>	<code>AND</code>	<code>!</code>

Exemple 5.2 Les instructions

```
3==3 or 9>24; 3==3 and 9>24; !(3==3)
```

renvoient respectivement `True`, `False` et `False`.

5.2 Les structures d'embranchements

Lorsque l'on veut exécuter des instructions seulement si une condition est vérifiée ou non, on utilisera une structure d'embranchement.

5.2.1 La structure **si ... alors ...**

La structure algorithmique

```
si une condition est vraie alors  
    bloc d'instructions  
fin si
```

s'écrit

```
if (condition)  
{  
    bloc d'instructions  
}
```

Exemple 5.3 Le code PHP suivant permet d'afficher TRUE si le nombre \$x est positif :

```
if ($x >= 0)  
{  
    echo 'TRUE';  
}
```

5.2.2 La structure **si ... alors ... sinon ...**

La structure algorithmique

```
si une condition est vraie alors  
    bloc d'instructions 1  
sinon  
    bloc d'instructions 2  
fin si
```

s'écrit

```
if (condition)  
{  
    bloc d'instructions 1  
}  
else  
{  
    bloc d'instructions 2  
}
```

Exemple 5.4 Le code PHP suivant permet d'afficher négatif ou positif suivant le signe de \$x :

```
if ($x <= 0)  
{  
    echo 'négatif';  
}  
else  
{  
    echo 'positif';  
}
```


5.2.3 Les structures d'embranchements multiples

- ★ La structure algorithmique

```
si condition 1 est vraie alors
    bloc d'instructions 1
sinon si condition 2 est vraie alors
    bloc d'instructions 2
... (on peut mettre autant de sinon si que nécessaire)
sinon
    dernier bloc d'instructions
fin si
```

s'écrit

```
if (condition 1)
{
    bloc d'instructions 1
}
else if (condition 2)
{
    bloc d'instructions 2
}
...
else
{
    dernier bloc d'instructions
}
```

Exemple 5.5 Le code PHP suivant permet d'afficher le signe de \$x :

```
if ($x<=0)
{
    echo 'négatif';
}
else
{
    echo 'positif';
}
```

- ★ Le langage PHP admet également une autre structure d'embranchement appelée switch :

```
switch (variable ou expression)
{
    case valeur 1:
        bloc d'instructions 1
        break;
    case valeur 2:
        bloc d'instructions 2
        break;
    ...
    case valeur n:
        bloc d'instructions n
        break;
    default: // optionnel
        bloc d'instructions
}
```

Lorsque la structure switch est utilisée dans une fonction qui a besoin de renvoyer une valeur, on utilise la commande `return` dans les blocs d'instructions.

Exemple 5.6 Etant donné une variable `$fruit`, afficher 25 si `$fruit='Orange'`, afficher 40 si `$fruit='Pomme'` et afficher 0 dans tous les autres cas.

```
switch ($fruit)
{
    case 'Orange':
        echo '25';
        break;
    case 'Pomme':
        echo '40';
        break;
    default:
        echo '0';
}
```

5.3 La structure de boucle pour

La structure

```
pour i de m à n faire
    bloc d'instructions
fin pour
```

s'écrit

```
for ($i=$m; $i<=$n; $i=$i+1)
{
    bloc d'instructions
}
```

Remarque 5.7 L'instruction `$i=$i+1` correspond à l'incrément du compteur de boucle `$i`. Avec cette instruction, l'incrément est de 1. On peut bien sûr la modifier et effectuer des incréments de 2, 3, voire négative.

Exemple 5.8 Parcours d'une chaîne de caractères sur les index :

```
$texte = 'Bonjour';
for ($i=0; $i<strlen($texte); $i++)
{
    echo $texte[$i];
}
```

Remarque 5.9 On peut parcourir de la même façon un tableau indexé numériquement. Pour les tableaux associatifs, on peut également les parcourir sur les clés avec la boucle itérative `foreach`.

5.4 La structure de boucle tant que

La structure

```
tant que une condition reste vraie faire
    bloc d'instructions
fin tant que
```

s'écrit

```
while(condition)
{
    bloc d'instructions
}
```

Exemple 5.10 Parcours d'un tableau indexé numériquement :

```
$tab = [3, 2, 1, 'Partez !'];  
$i = 0;  
while ($i < sizeof($tab))  
{  
    echo $tab[$i]. "<br/>";  
    $i++;  
}
```

6 Les fonctions

Les fonctions PHP se manipulent comme les fonctions Python ou JavaScript. Leur seule différence réside dans la syntaxe. En PHP, la syntaxe d'une fonction est la suivante :

```
function nom_de_la_fonction(liste_paramètres )  
{  
    instruction 1;  
    ...  
    instruction n;  
    return liste_valeurs  
}
```

Remarque 6.1

- ★ La liste des paramètres peut être vide.
- ★ La fonction peut ne pas renvoyer de valeurs (on parle alors de *procédure*).
- ★ Comme dans les autres langages de programmation, une fonction peut faire appel à plusieurs *sous-fonctions*.

Remarque 6.2 La fonction `function_exists()` renvoie TRUE si la fonction existe et FALSE sinon.

7 Liaisons entre les fichiers PHP

Pour éviter d'avoir des fichiers trop volumineux (beaucoup de lignes de codes) ou simplement pour aérer son code ou la gestion d'un site web, il est souvent utile d'écrire plusieurs fichiers séparés et de les inclure au moment voulu pour pouvoir les utiliser. Pour se faire, on dispose des fonctions suivantes :

- ★ `require()` : permet d'inclure le contenu d'un autre fichier appelé, et provoque une erreur bloquante s'il est indisponible ;
- ★ `require_once()` : même chose que `require()`, mais ne le fait qu'une seule fois en tout et pour tout dans le même document, si `require()` a déjà été appelé auparavant avec le même nom de fichier ;
- ★ `include()` : permet d'inclure le contenu d'un autre fichier appelé, mais ne provoque pas d'erreur bloquante s'il est indisponible ;
- ★ `include_once()` : même chose que `include()`, mais ne le fait qu'une seule fois en tout et pour tout dans le même document si `require()` a déjà été appelé auparavant avec le même nom de fichier.

Remarque 7.1 Le paramètre `nom_fichier` est obligatoire et est une chaîne de caractères correspond au chemin relatif depuis le fichier PHP appelant.

8 Dates et heures

★ Gestion des fuseaux horaires.

- `date_default_timezone_set(chaine)` : définit le décalage horaire par défaut de toutes les fonctions donnant la date et l'heure suivant un fuseau horaire donné par la chaîne de caractères `chaine`.

Exemple 8.1 L'instruction

```
date_default_timezone_set('Europe/Paris');
```

indique que toutes les fonctions donnant la date et l'heure qui seront utilisées ultérieurement seront basées sur l'heure de Paris.

★ Affichage de la date et de l'heure du serveur local. (serveur sur lequel se situe le fichier PHP utilisé)

- `date("d/m/Y")` : affiche la date du serveur local sous la forme JJ/MM/AAAA.
- `date("G\hi")` : affiche l'heure du serveur local sous la forme HH:mm.

Remarque 8.2 Il existe un grand nombre de formatage pour les dates et les heures. On renvoie à internet pour plus de détails.

★ Formatage d'une date.

- `date_create_from_format(form, chaine)` : renvoie un objet représentant la date et/ou l'heure spécifiées par la chaîne de caractères `chaine` et formatées dans le format `form` donné.

Exemple 8.3 Les instructions

```
$date = '12:3:2019';  
$date_formatee = date_create_from_format('j:n:Y', $date);
```

permettent d'affecter à la variable `$date_formatee` la date `$date` sous une forme `j:m:aaaa`.

Il existe de nombreux formatages possibles. Ceux-ci permettent ensuite de récupérer diverses informations à l'aide de la fonction suivante.

- `date_format($date, $form)` : renvoie un élément particulier de la date formatée `$date` obtenue à partir de la fonction `date_create_from_format()` et donné par la chaîne de caractères `$form` : par exemple,
 - 'o' donne le numéro de l'année au format ISO-8601,
 - 'W' donne le numéro de la semaine (démarre un lundi) au format ISO-8601,
 - 'N' donne le numéro du jour (1 pour lundi, ..., 7 pour dimanche) au format ISO-8601,
 - 'G' donne l'heure entre 0 à 23.

Il existe bien sûr plusieurs autres types de format. On renvoie à internet pour plus de détails.

★ Création de date.

- `date_create()` : renvoie un objet `DateTime` afin de manipuler les fonctions ci-dessous.
- `date_isodate_set($obj, $y, $w)` : renvoie dans l'objet `obj` créé avec la fonction `date_create()` une date selon la norme ISO-8601 définie par l'année `y` et la semaine `w`.
- `date_add($date, new DateInterval($chn))` : ajoute à la date `date` une durée définie par la chaîne de caractères `chn`. Cette chaîne commence obligatoire par la lettre P (pour *period*) et est constituée de différentes durées représentées par un nombre entier naturel suivie d'une désignation de période : Y pour année, M pour mois, D pour jour, H pour heure, M pour minute et S pour seconde. Pour l'utilisation des trois dernières périodes, il faut spécifier la portion avec la lettre T.

Exemple 8.4 Deux jours sera 'P2D' ; deux secondes sera 'PT2S' ; six années et cinq minutes sera 'P6YT5M'

9 Gestion des formulaires

Lorsque l'on soumet les informations d'un formulaire à un script PHP, celui-ci récupère les données du formulaire sous forme de variables, dont le nom dépend, bien sûr, de la méthode d'envoi GET ou POST.

Méthode d'envoi :	GET	POST
Nom des variables :	<code>\$_GET ['nom_du_champ']</code>	<code>\$_POST ['nom_du_champ']</code>

Exemple 9.1 Un formulaire contient un champ dont le nom est donné par `name='pays'`. Si les informations de ce formulaire sont envoyées avec la méthode GET, alors la variable `$_GET [ 'pays' ]` récupère le contenu du champ `pays`. Dans la pratique, on écrira souvent `$p=$_GET [ 'pays' ] ;` afin d'avoir un nom de variable plus simple à utiliser.

Les variables `$_GET` et `$_POST` sont des tableaux associatifs dont les clés sont les noms `name` des différents champs envoyés par les formulaires et les valeurs associées sont les valeurs `value` correspondantes.

- ★ Toutes les valeurs reçues sont des chaînes de caractères. Il est donc important de vérifier le type de données reçues (numérique, alphanumérique, vide, ...) car PHP procède à du transtypage automatiquement pour effectuer des calculs ou autres.
- ★ Il est également important de vérifier, dans le cas des boutons radios et des cases à cocher, si la ou les clé(s) correspondantes existe(nt) dans le tableau car ceux-ci n'envoient rien s'ils n'ont pas été cochés. Pour cela, on peut utiliser la fonction `isset (nom_variable)` qui renvoie `TRUE` si la variable à analyser existe et `FALSE` sinon.